

Real-Time Schedulers for STM32: Performance Evaluation of EDF over SuperLoop

Vladimir Toporkov, Artem Burmyakov
Innopolis University

ToDo

1. AB: To introduce one paragraph per each point: . . . 1
2. AB: Can we spell “SuperLoop” instead of “Super Loop”? 1
3. AB: to create a .bib file and add reference 1
4. AB: to put reference to Burmyakov2026. 1
5. AB: Can we avoid using the “scheduled release” term? Could we use “the release time of the j -th job of the task τ_i ” or similar instead? 1

I. INTRODUCTION

1.AB: To introduce one paragraph per each point:

- 1) STM32 microcontroller;
- 2) Tasks from peripheral sensors;
- 3) Real-time scheduling: SuperLoop and EDF.

2.AB: Can we spell “SuperLoop” instead of “Super Loop”?

A. Contribution

We evaluate the scheduling performance of EDF over the default Super Loop scheduler for STM32 systems. Our implementations for both schedulers are publicly available.

II. BACKGROUND ON STM32

We illustrate the hardware organization for our system in Fig. 1. The central processing unit is an STM32F767 uniprocessor, which in turn is connected to four sensors:

- DHT11: to measure temperature and humidity;
- HC-SR04: to measure distance;
- OV2640: to capture images;
- SPW2430: to acquire acoustic signal.

III. SYSTEM MODEL

We rely on a classical real-time periodic system model by Liu and Layland 3.AB: to create a .bib file and add reference. We reuse the system model notation from 4.AB: to put reference to Burmyakov2026.

5.AB: Let us reuse the notation from that paper.

A real-time system $\mathcal{T}$ is modeled by a set of n periodic tasks $\{\tau_1, \dots, \tau_n\}$, where each task $\tau_i = (C_i, D_i, P_i)$ is modeled by the worst-case execution time C_i , a period P_i between the releases of consecutive jobs, and a relative implicit deadline $D_i = P_i$. Also, the quantity $U_i = C_i/P_i$ is the task utilization and $U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$ is the utilization of the entire task set $\mathcal{T}$. Each task $\tau_i \in \mathcal{T}$ releases a potentially infinite sequence

of jobs of the period P_i : if τ_i releases its j -th job at time $r_{i,j}$, then it releases its next job exactly at time $r_{i,j+1}$, such that

$$r_{i,j+1} = r_{i,j} + P_i. \quad (1)$$

Any j -th job of τ_i released at $r_{i,j}$ requires at most C_i units of CPU time and must complete not later than the absolute deadline $r_{i,j} + D_i$, otherwise the job is said to miss a deadline.

6.AB: Can we avoid using the “scheduled release” term? Could we use “the release time of the j -th job of the task τ_i ” or similar instead?

Release indices include skipped activations, for which no completion time is defined.

- A schedule specifies which job executes on the processor at each time instant, or whether the processor executes no job.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job J_i^j has missed its deadline if $f_i^j > d_i^j$.
- The response time of a completed job J_i^j is $f_i^j - R_i^j$, the time from its scheduled release to completion.

Table I summarizes the notation introduced above.

IV. SCHEDULING PRINCIPLES

A. Super Loop scheduler

The considered Super Loop scheduler repeatedly scans tasks in a fixed order and executes each selected job non-preemptively. To illustrate its behavior, let us consider the two periodic tasks in Table II. Both tasks first release at $t = 0$.

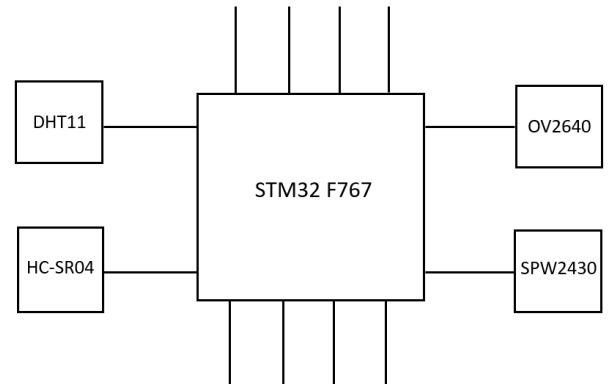
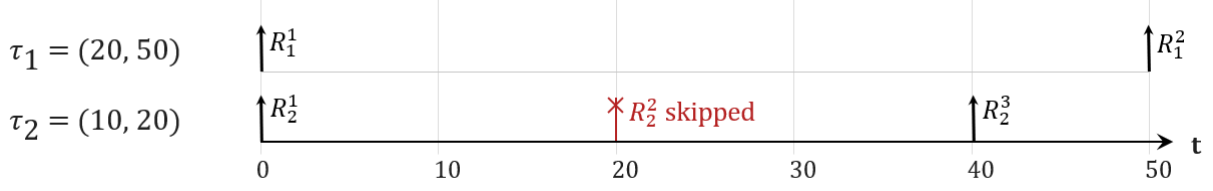


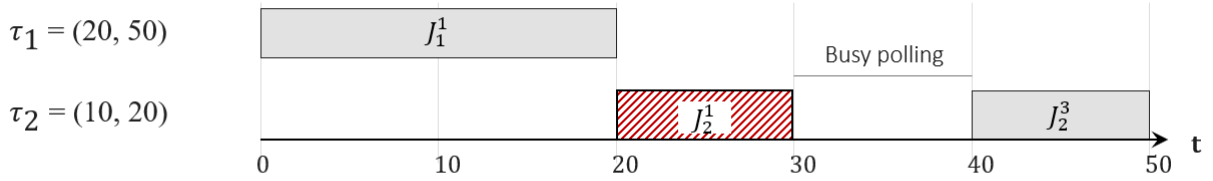
Fig. 1. Our experimental hardware platform.

$$\tau_i = (C_i, D_i = T_i)$$

Scheduled Releases



Super Loop execution scenario



Absolute Deadline: 20 Completion: 30

J_j^i – job execution R_j^i – scheduled release time

Fig. 2. Super Loop execution scenario for the task set in Table II.

TABLE II
TASK SET USED IN THE SUPER LOOP EXAMPLE.

Task	C_i	T_i	D_i
τ_1	20	50	50
τ_2	10	20	20

The resulting Super Loop schedule is shown in Fig. 2.

Let t^{cur} denote the current system time and t_i^{next} the next scheduled release time of task τ_i . During execution of J_i^j , the release variable t_i^{next} retains R_i^j . At completion, f_i^j denotes the completion time, and a deadline miss is recorded if $f_i^j > d_i^j$. The scheduler then advances t_i^{next} by T_i . While $t_i^{next} < f_i^j$, the scheduled release at t_i^{next} is counted as skipped and t_i^{next} is advanced by T_i again. A release at $t_i^{next} = f_i^j$ is

TABLE I
SYSTEM MODEL NOTATION.

Symbol	Meaning
$\mathcal{T}, n$	Task set and number of tasks
τ_i	Task i
C_i	Execution time of a job of τ_i
T_i	Task period
$D_i = T_i$	Relative deadline, equal to the period
J_i^j	Job associated with the j -th scheduled release of τ_i
R_i^j	Scheduled release time
f_i^j	Completion time of an executed job
$d_i^j = R_i^j + D_i$	Absolute deadline
$U = \sum_{i=1}^n C_i/T_i$	Task-set utilization

retained.

Algorithm 1 summarizes the illustrated scheduling logic. Since $R_i^1 = 0$, the release index is $j = 1 + t_i^{next}/T_i$, including skipped releases. The clock used by NOW starts at zero.

The corresponding scheduling state is shown in Table III.

TABLE III
SUPER LOOP SCHEDULING STATE FOR FIG. 2.

t^{cur}	t_1^{next}	t_2^{next}	Deadline miss check	Release skip check
0	0	0	–	–
20	50	0	$20 < R_1^1 + 50$	$20 < R_1^2$
30	50	40	$30 > R_2^1 + 20$	$30 > R_2^2$
40	50	40	–	–
50	50	60	$50 < R_2^3 + 20$	$50 < R_2^4$

In Fig. 2, J_1^1 executes from $t = 0$ to $t = 20$. Then, J_2^1 executes until $t = 30$, missing its deadline at $t = 20$. At completion, the release scheduled for $R_2^2 = 20$ is counted as skipped and t_2^{next} advances to $R_2^3 = 40$. Thus, no J_2^2 is executed.

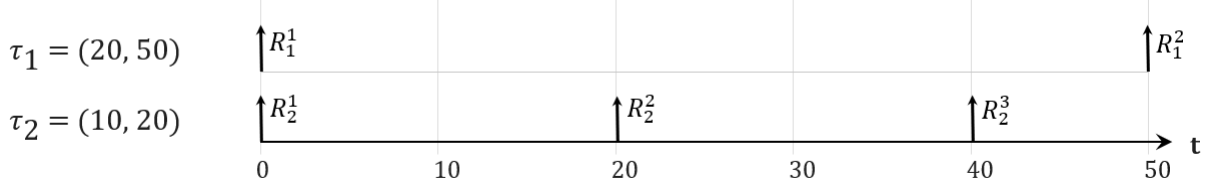
B. Earliest Deadline First

Unlike Super Loop, Earliest Deadline First (EDF) selects jobs according to their absolute deadlines.

We first consider preemptive EDF with immediate preemption. At each scheduling decision, the ready job with the earliest absolute deadline d_i^j is selected (Section III). If a newly

$$\tau_i = (C_i, D_i = T_i)$$

Scheduled Releases



EDF execution scenario

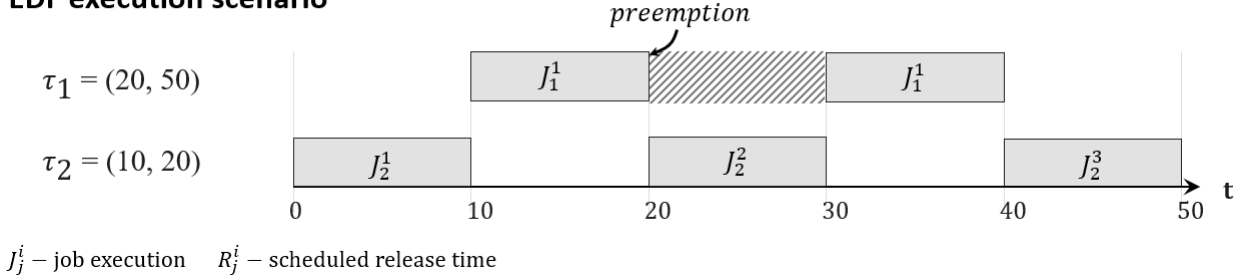


Fig. 3. EDF execution scenario for the task set in Table II. Hatching indicates a preempted job awaiting resumption.

Algorithm 1 Super Loop Scheduler

```

1: function SUPERLOOP( $\mathcal{T}$ )
2:   for all  $\tau_i \in \mathcal{T}$  do
3:      $t_i^{next} \leftarrow 0$ 
4:   end for
5:   while true do
6:     for  $i \leftarrow 1$  to  $n$  do ▷ Fixed task order
7:        $t^{cur} \leftarrow \text{NOW}$ 
8:       if  $t^{cur} \geq t_i^{next}$  then
9:          $j \leftarrow 1 + t_i^{next}/T_i$ 
10:         $R_i^j \leftarrow t_i^{next}$ 
11:        EXECUTE( $J_i^j$ ) ▷ Non-preemptive
12:         $f_i^j \leftarrow \text{NOW}$ 
13:        if  $f_i^j > R_i^j + D_i$  then
14:          RECORDMISS( $J_i^j$ )
15:        end if
16:         $t_i^{next} \leftarrow t_i^{next} + T_i$ 
17:        while  $t_i^{next} < f_i^j$  do
18:          RECORDSKIP( $\tau_i, t_i^{next}$ )
19:           $t_i^{next} \leftarrow t_i^{next} + T_i$ 
20:        end while
21:      end if
22:    end for
23:  end while
24: end function

```

released job has an earlier absolute deadline than the running job, it preempts the running job.

Fig. 3 illustrates EDF scheduling for the task set in Table II.

At $t = 20$, J_2^2 is released with $d_2^2 = 40 < d_1^1 = 50$ and preempts J_1^1 . After J_2^2 completes at $t = 30$, J_1^1 resumes and

completes at $t = 40$.

Chunked EDF is used as a simplified prototype implementation, avoiding the additional complexity of full task-context switching during initial evaluation. Synthetic jobs execute in chunks of maximum duration Δ , with job selection repeated after each chunk. An earlier-deadline job released during a chunk may therefore wait until the chunk completes. Sensor jobs use separate staged handlers.

For illustration, Fig. 4 uses $\Delta = 4$ ms. Job J_2^2 is released at $t = 20$ ms and preempts J_1^1 at the next chunk boundary $t = 22$ ms. Thus, the preemption delay is $\delta = 2$ ms. The final chunk of J_2^2 lasts 2 ms, since the preceding two chunks account for 8 ms of its execution time $C_2 = 10$ ms.

Ignoring scheduling overhead, the delay until the next chunk boundary satisfies $0 \leq \delta \leq \Delta$.

Algorithm 2 summarizes chunked EDF for synthetic jobs, assuming first releases at $t = 0$ as in the example. The set $\mathcal{R}$ contains jobs admitted for execution but not yet completed. Initially, $\mathcal{R} = \emptyset$, $t_i^{next} = 0$, and no job is active. Equal deadlines are resolved by task order. EXECUTECHUNK executes at most Δ of the selected job's remaining execution and preserves its progress.

RELEASEJOB creates the job associated with t_i^{next} , sets its release and absolute deadline, and adds it to $\mathcal{R}$ as active. COMPLETEJOB checks its deadline and advances t_i^{next} using the skip rule in Algorithm 1, then removes the job from $\mathcal{R}$ and marks the task inactive.

V. PERFORMANCE EVALUATION

A. Experimental setup

Experiments run on the NUCLEO-F767ZI at 96 MHz. Physical sensors are disabled in these experiments. Each synthetic

Algorithm 2 Chunked EDF Scheduler

```

1: function CHUNKEDEDF( $\mathcal{T}, \Delta$ )
2:   while true do
3:      $t^{cur} \leftarrow \text{NOW}$ 
4:     for all  $\tau_i \in \mathcal{T}$  do
5:       if  $t_i^{next} \leq t^{cur}$  and  $\tau_i$  has no active job then
6:         RELEASEJOB( $\tau_i, t_i^{next}$ )
7:       end if
8:     end for
9:     if  $\mathcal{R} \neq \emptyset$  then
10:       $J_i^j \leftarrow \text{SELECTEDF}(\mathcal{R})$ 
11:      EXECUTECHUNK( $J_i^j, \Delta$ )
12:      if COMPLETED( $J_i^j$ ) then
13:         $f_i^j \leftarrow \text{NOW}$ 
14:        COMPLETEJOB( $J_i^j, f_i^j$ )
15:      end if
16:    end if
17:  end while
18: end function

```

job executes a timed busy-wait workload, the scheduler also uses busy polling. Timing is obtained from the DWT cycle counter, and UART reports are emitted after measurement stops.

The results below use three tasks with $(T_1, T_2, T_3) = (10, 50, 200)$ ms and $D_i = T_i$. Table IV lists the configured execution budgets. Thus, U denotes nominal workload utilization and excludes instrumentation and scheduling overhead. Both schedulers use the same task set, Super Loop scans tasks in index order. For clarity, the illustrative schedules assume synchronous first releases at $t = 0$, whereas the implementation releases each task for the first time one period after initialization, $R_i^1 = t_0 + T_i$.

The main comparison uses 60 s windows and $\Delta = 1$ ms for EDF. The chunk-size study uses $\Delta \in \{1, 2, 4\}$ ms at $U \in \{50, 90, 100\}\%$, the figure below shows $U = 90\%$. Each configuration is run three times from reset. Additional campaigns use two tasks and 100 s windows to examine task-set and window-length effects. Extended diagnostic statistics are disabled in all these campaigns.

TABLE IV

THREE-TASK SYNTHETIC WORKLOADS. EXECUTION BUDGETS ARE IN μs .

Nominal U (%)	C_1	C_2	C_3
50	1667	11111	22222
65	2167	14444	28889
75	2500	16667	33332
80	2667	17778	35556
90	3000	20000	40000
95	3167	21111	42222
100	3333	22222	44444

B. Evaluation metrics

Let N_i , M_i , and S_i be the numbers of completed jobs, completed jobs that missed their deadlines, and skipped releases of τ_i in a run. Let W be the requested measurement-window duration. We report:

- Deadline-miss ratio: $100M_i/N_i$ percent, defined for $N_i > 0$.
- Skipped-release ratio: $100S_i/(W/T_i)$ percent, normalized by the nominal number of releases in the requested window.
- Average response time: the mean of $f_i^j - R_i^j$ over completed jobs. The maximum observed response time is also recorded.
- Selection overhead: measured time spent selecting jobs in iterations that execute work, divided by the actual measurement duration, in percent. This excludes uninstrumented scheduler work and is not total processor utilization.

Measurement stops at the first loop boundary after W , its actual duration may therefore be slightly longer. Released but unfinished tracked jobs and their overdue status are recorded separately and excluded from completion-based metrics. Subsequent releases awaiting skip accounting are not included in this pending-job count. Plot points are arithmetic means of the corresponding per-run metrics over three runs, not pooled averages over all jobs.

C. Super Loop and EDF comparison

Fig. 5 compares the schedulers for τ_1 . Under Super Loop, its deadline-miss and skipped-release ratios generally increase

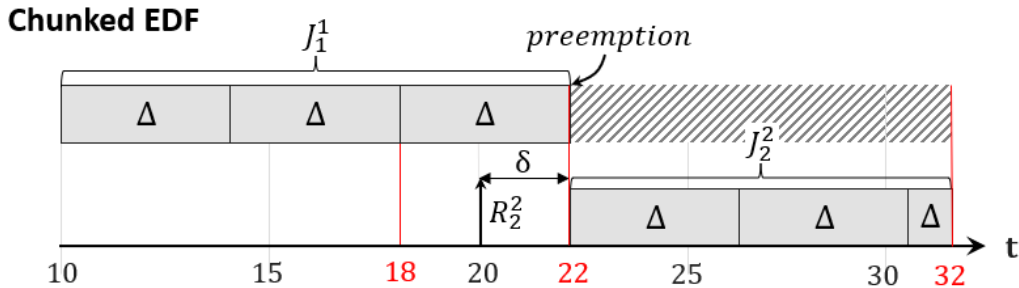


Fig. 4. Chunk-bounded preemption in the EDF implementation.

with utilization, as does its average response time. With chunked EDF ($\Delta = 1$ ms), no deadline misses or skipped releases are recorded for τ_1 in the evaluated three-task configurations. Selecting jobs by absolute deadline reduces the blocking caused by the fixed Super Loop order.

Fig. 6 shows the corresponding results for τ_3 . Its average response time under chunked EDF increases with utilization as jobs with earlier absolute deadlines execute first. At nominal $U = 100\%$, no capacity is reserved for scheduling and execution overhead, deadline misses and skipped releases are observed for τ_3 . The lower response time of τ_3 under Super Loop occurs alongside skipped releases of τ_1 , so it does not imply better service for the entire task set.

D. Effect of chunk duration

Fig. 7 examines Δ at $U = 90\%$. Larger chunks reduce the frequency of scheduling decisions. In these measurements, the reported selection overhead decreases, while the average response time of τ_1 increases and that of τ_3 decreases. Longer non-preemptive intervals delay τ_1 , the result for τ_3 also reflects the changed execution order and fewer scheduling interruptions. The overhead measurements remain preliminary pending validation of the instrumentation.

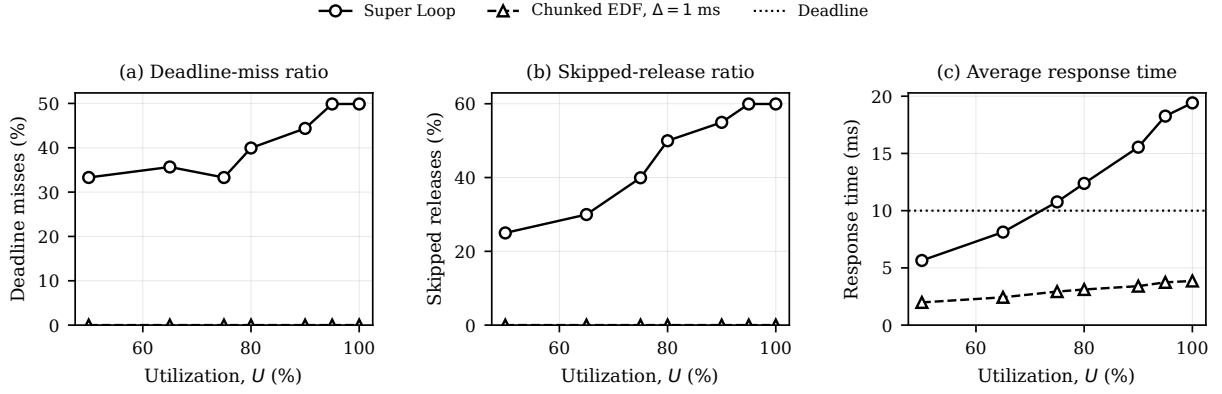


Fig. 5. Performance for τ_1 : three synthetic tasks, $\Delta = 1$ ms for EDF, 60 s windows.

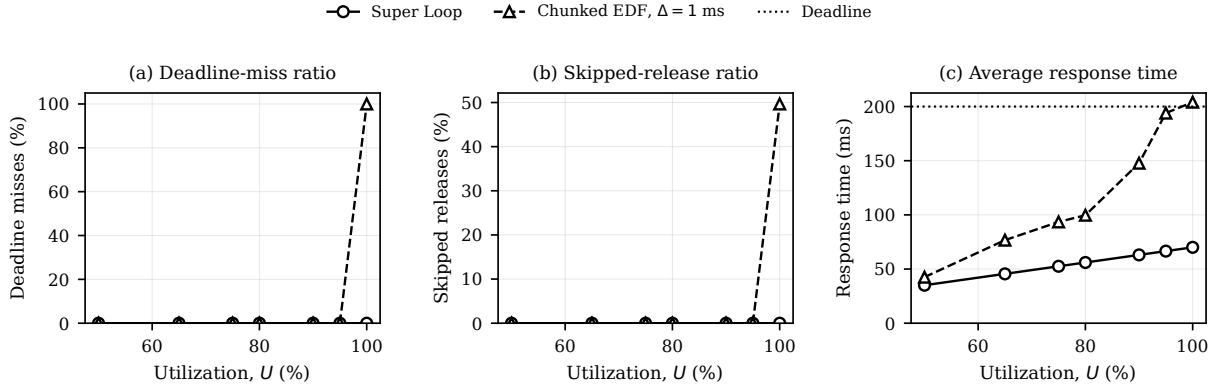


Fig. 6. Performance for τ_3 under the same conditions as Fig. 5.

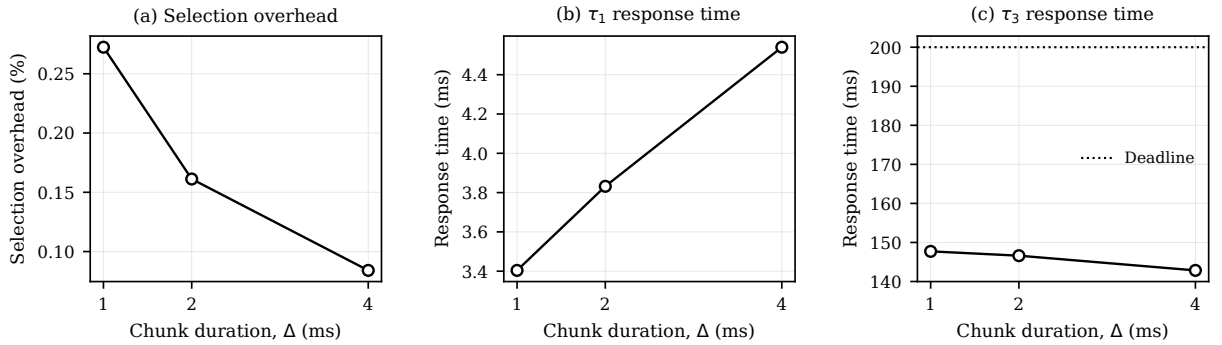


Fig. 7. Impact of chunk duration at $U = 90\%$ with three synthetic tasks and 60 s windows.